ARI: An Autonomous Research Infrastructure

Takanori Kotama

v0.8.1, 2026-06-01

Abstract

We describe **ARI**, an autonomous research infrastructure that couples a tree-search-based exploration loop with a paper-generation pipeline. The exploration loop is a best-first tree search (BFTS) over research-step nodes; expansion uses a Reason-and-Act (ReAct [1])-style agent that issues tool calls into a registry of fourteen domain skills. The paper pipeline turns the surviving lineage into a draft, then iteratively refines it under a hierarchical rubric. Every artefact lives inside a single checkpoint directory, which is the unit of reproducibility, lineage, and cost accounting. This report formalises the algorithms and the design rationale, reports preliminary observations, and discusses the determinism / novelty trade-off that constrains the design.

1 Introduction

1.1 Motivation

Research workflows in machine learning and adjacent fields share a common skeleton: brainstorm a research idea, design and run an experiment, analyse the result, write a paper, and iterate. Each step is increasingly tractable for large language models, but stitching them into a single autonomous loop has, so far, been an open engineering problem. The published systems that come closest — AIDE, the AI Scientist family, VirSci, and PaperBench evaluators — each fix one or two of these axes while leaving the others manual.

ARI exists to fill that gap. We assume the user supplies a research question and a compute budget; the system returns a publishable paper, the experimental code that backs every claim, and a complete audit trail of how it got there. The audit trail is non-negotiable: every node of the search tree, every tool call, every model invocation, and every dollar of API spend is recorded inside a single *checkpoint directory* ([2] adopt a similar discipline). The checkpoint is the unit of reproducibility.

1.2 Contributions

This report formalises three contributions:

1. A **best-first tree search** over research-step nodes whose selection score $U(n)$ separates the empirical reward, a novelty term, and a depth penalty (section 3.1).
2. A **paper-generation pipeline** keyed off a hierarchical rubric R whose leaf judges grade individual claims; the system score is $\text{score}(P) = \sum_i w_i \, \text{judge}_i(P)$ (section 4.3). The PaperBench replicator is integrated as a tool call (section 4.5).
3. A **checkpoint scoping** discipline that makes the pipeline fully reproducible: every external state (memory, cost ledger, lineage pointers) is written below `$ARI_CHECKPOINT_DIR`, never the user's home directory. This is the operational consequence of the *determinism* design principle, one of five that section 2.5 lays out in full (we refer to it as P2 there and in the rest of the report).

1.3 Scope of this report

This report describes `ari-core` and the fourteen `ari-skill-*` packages at version v0.8.0, which promotes the PaperBench reproduction path to a protocol-faithful three-stage contract — agent rollout, reproduction, and grading — adds host-truthful environment guardrails that stop the rollout agent from assuming capabilities the host does not provide, makes the search-evaluation layers configurable, and carries out a first operational reproduction pass on a real lossy-compression target (section 5). The body describes the algorithms and data flow at the module level; the verbatim LLM prompts the system uses are listed in section B. The empirical study is preliminary at this version — section 5 reports the observations available so far, and a controlled benchmark is left to future work.

1.4 Organisation

Section 2 gives a top-down view of ARI: the orchestrator, the fourteen skills, the pipeline DAG, and the design principles. Sections 3 and 4 are the two technical cores, each with its own formal definitions and empirical observations. Section 5 reports preliminary observations; section 6 positions the work; section 7 collects open problems.

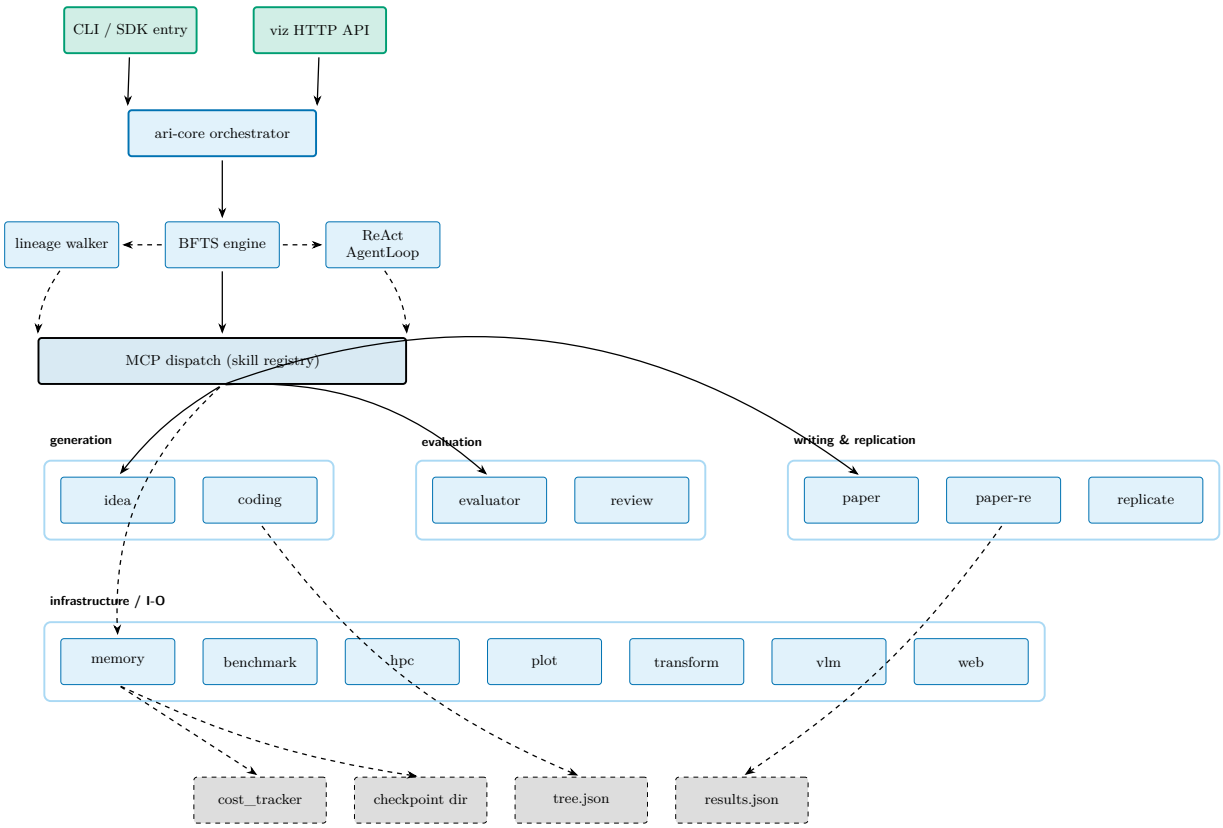

Figure 1: The full ARI stack at v0.8.0. The CLI / SDK entry-points drive a single orchestrator (`ari-core`); the orchestrator owns three engines (BFTS, lineage walker, ReAct `AgentLoop`) and dispatches into the 14-skill registry through the MCP layer. All persistence lands inside one checkpoint directory: cost ledger, `tree.json`, and `results.json`. Solid edges are control; dashed edges are state.

2 System Overview

2.1 The full stack

Figure 1 draws the full ARI stack at v0.8.0. The system splits into one driver and many tool providers. The driver (`ari-core`) owns the orchestrator, the BFTS engine, the lineage walker, the agent loop, the checkpoint, the cost tracker, and the configuration loader. Each skill is a separate Python package shipped as a *Model Context Protocol* (MCP) server with its own `skill.yaml` declaration. MCP is the JSON-RPC-over-stdio interface that Anthropic standardised for exposing tool-style capabilities to an LLM agent loop; ARI uses it as the universal seam between `ari-core` and every skill. The fourteen skills currently in tree are: `benchmark`, `coding`, `evaluator`, `hpc`, `idea`, `memory`, `orchestrator`, `paper`, `paper-re`, `plot`, `replicate`, `transform`, `vlm`, and `web`.

The orchestrator dispatches calls but does not perform domain work itself. This separation matters because skills can be added or swapped without touching the search algorithm: the BFTS engine is unaware of which skills participate; everything is mediated by the MCP dispatch layer.

The smaller diagram in fig. 2 below is the "operational" view (control + state outputs) used in the rest of the report; fig. 1 is the "layered" view (driver → MCP → skills → persistence).

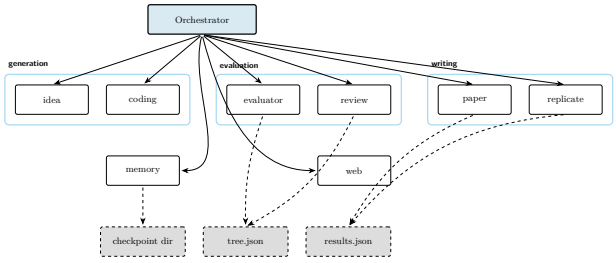

Figure 2: Operational view: orchestrator dispatches into the skill registry; every skill writes only into the active checkpoint directory. Dashed edges indicate state.

2.2 Pipeline DAG

A run reduces to a four-station pipeline (fig. 3). The idea station selects or generates research questions; the exploration station spawns nodes under BFTS; the paper station compiles the best lineage into a draft; and the review station grades the draft against the rubric. The graph is a DAG plus one back edge from review to paper: only this edge can introduce non-monotonicity (refining worsens an axis), and the *convergence criterion* (section 4.4) is what bounds the back-edge count.

The pipeline is driven by a single runner in the orchestrator; a scientific-data assembler closes the gap between exploration output and paper input, collating the per-node artefacts the writer needs.

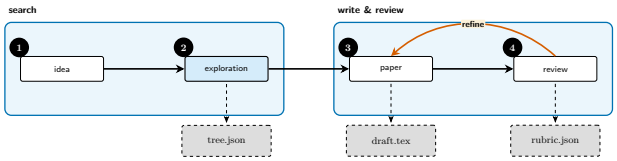

Figure 3: Pipeline DAG: idea → exploration → paper → review, with *refine* closing the back edge. Dashed outputs are persisted to the checkpoint; the refine cycle rewrites the draft and re-runs the review.

2.3 BFTS control flow

Figure 4 traces a single BFTS step end-to-end. The implementation is laid out in section 3; the diagram fixes the vocabulary used by the rest of the report (frontier, selection, fallback, expand, prune, stagnation) and the prompt files (`bfts_select.md`, `bfts_expand_select.md`, `bfts_expand.md`) that materialise each LLM-driven step.

2.4 paper-re component view

Figure 5 is the second-level view of `paper-re`. ARI's contribution at this layer is small but specific: a thin adapter wraps each agent tool so its output is size-bounded and redirects the vendored solver's hard-coded paths to ARI's per-node workspace; everything else flows through PaperBench upstream verbatim, which keeps ARI aligned with the upstream replication-task framing. Section 4.5 develops this integration.

2.5 Checkpoint scoping and the design principles

ARI is built around five *design principles* (P1–P5). The binding one is **P2: determinism**. Every randomised step records its seed; every LLM call records its model identity, prompt, and output; every tool call is tagged with its node identity (a copy-on-write guard) so its writes land inside the right node's working tree. Re-running a checkpoint must reproduce the same artefacts byte-for-byte, except for fields explicitly marked as wallclock or as model-side non-determinism.

The other principles, in summary:

- **P1 single artefact tree**: every output of a run lives below `$ARI_CHECKPOINT_DIR`. Nothing leaks to `$HOME` or `/tmp`.
- **P3 small components**: each skill is a separate package; replacing one skill does not require rebuilding `ari-core`.
- **P4 explicit lineage**: every node and every checkpoint record their parent, so a derivative experiment can recompute only its delta.
- **P5 cost transparency**: the cost tracker attaches dollar costs to every model call so the user can bound a run.

The checkpoint serialiser writes three top-level files: the full search tree to `tree.json`, per-node payloads to `nodes_tree.json`, and end-of-run aggregates to `results.json`. Lineage walks across checkpoints climb from a child to its ancestors.

Figure 6 draws the on-disk shape of a checkpoint: shared run-level files on the left, per-node working trees on the right. The shape is the contract: any tool that takes ARI's output as input (a follow-up run, a regression check, a paper draft) points at a single such directory.

3 Exploration Algorithm

3.1 Formalisation

The exploration loop maintains a tree $\mathcal{T} = (\mathcal{N}, E)$ in which each node $n \in \mathcal{N}$ represents a single research step: an idea revision, a code change, a benchmark run, an analysis, etc. A node carries

- a discrete state $s(n) \in \{\text{open}, \text{eval}, \text{done}, \text{failed}\}$,
- a categorical *label* drawn from the `NodeLabel` enumeration,
- per-axis evaluation metrics in a freeform dict `node.metrics`, including a distinguished scalar `_scientific_score` $\in [0, 1]$,
- a Boolean `has_real_data` that flags whether the node carries actual measurements rather than provisional text, and
- the bookkeeping field `depth` surfaced both in the selection prompts and to the pruning predicate. ARI does not retry failed nodes, so no `retry_count` signal is surfaced.

The aggregation of axis-level signals into the scientific score is delegated to any implementation of the `Evaluator` protocol; ARI does not pin a fixed linear combination. The protocol is the only seam between BFTS and downstream rubric evaluation, which keeps the search engine agnostic about how axes get reduced to a scalar.

Run-loop state. Each iteration of the outer loop transforms the tuple

$$S = (\mathcal{F},\ \mathcal{P},\ e,\ H),$$

where $\mathcal{F} \subseteq \mathcal{N}$ holds completed nodes (done / failed) eligible for expansion, $\mathcal{P} \subseteq \mathcal{N}$ holds nodes in state open ready to execute, $e : \mathcal{N} \to \mathbb{N}$ counts how many times each frontier node has been expanded, and $H = (l_1, \ldots, l_t)$ is the sliding window of recently-executed labels (length capped at $W = 20$, see section 3.3). The initial state is $S_0 = (\emptyset, \{n_{\text{root}}\}, \mathbf{0}, ())$.

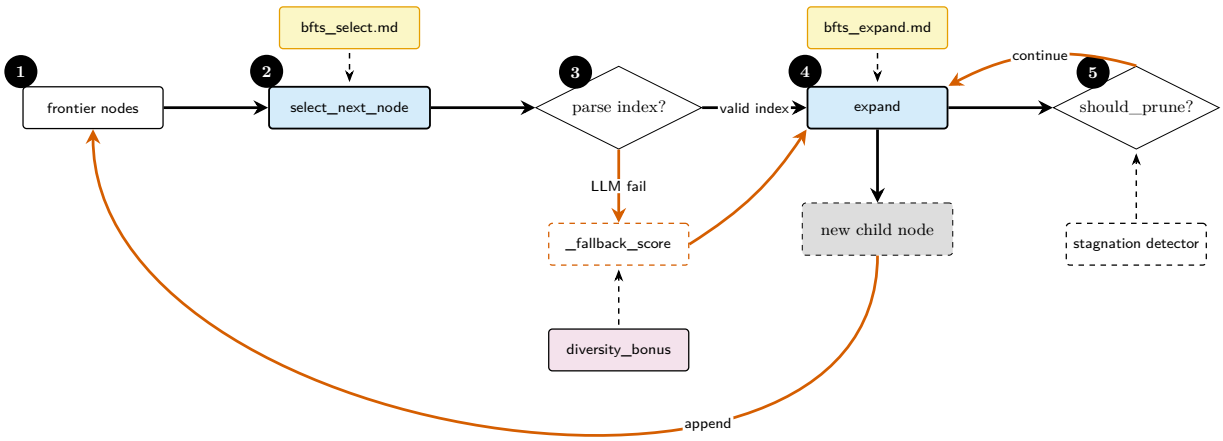

Figure 4: BFTS control flow. Two LLM-driven decision points (`select_next_node`, `expand`) each consult a separate prompt file; the selection decision falls back to a deterministic `_fallback_score` (which combines `_scientific_score` with `diversity_bonus`) when the LLM returns an unparseable index. Pruning is gated by both `should_prune` (max-total-nodes hard cut) and the stagnation detector. Detail in section 3.

Pruning predicate. The hard exploration budget is the predicate

$$\Pi(n;\,|\mathcal{N}|) \;=\; \mathbb{1}[|\mathcal{N}| \geq B_N] \;\vee\; \mathbb{1}[\mathrm{depth}(n) \geq B_D] \;\vee\; \sigma(n), \quad (1)$$

where B_N = `config.max_total_nodes`, B_D = `config.max_depth`, and the *sterile* predicate

$$\sigma(n) \;=\; \mathbb{1}[\,|\Delta_n| = 0\,] \quad (2)$$

(with Δ_n the set of files added, modified, or deleted by n) fires when a node finishes its agent loop without writing any new artefact relative to its parent. The run loop computes this file diff and records the result as a Boolean `_sterile` flag on the node; `should_prune` reads that flag together with the two caps. The caller passes the live $|\mathcal{N}|$ each iteration so there is exactly one source of truth for the node count. Step / cost budgets are enforced separately by the cost tracker at the call site, not inside the BFTS engine.

Retire predicate. On top of Π, the run-loop applies a softer *frontier retire predicate* ρ to drop a parent f from $\mathcal{F}$ once it is no longer the most productive lead:

$$\rho(f, c) \;=\; \underbrace{\mathbb{1}[r(c) > r(f)]}_{\text{Rule A}} \;\vee\; \underbrace{\mathbb{1}[e(f) \geq E_{\max}]}_{\text{Rule B}}, \quad (3)$$

with $E_{\max}$ = `config.max_expansions_per_node` (default 4). Rule A retires f as soon as a child c outscores it; Rule B caps the per-parent budget so the search spreads. ρ does *not* delete the node — its history stays on $\mathcal{T}$ — only its eligibility to be re-expanded is revoked.

The full step decomposition of one outer iteration (inner expansion sub-loop, parallel ReAct batch, post-execution retire) is shown in fig. 7; the explicit pseudocode is algorithm 1 in section 3.2.

3.2 Run-loop algorithm

Algorithm 1 writes the iteration as explicit pseudocode. The inner WHILE fills empty worker slots one expansion at a time (so workers always start running before the next expansion is proposed); the outer block then assembles a batch by calling `select_next_node` repeatedly — one node per call — until the batch reaches the worker cap, and executes that batch in parallel; the post-execution block (a) records the executed label into H so the diversity bonus reflects what actually ran, and (b) applies ρ Rules A and B.

3.3 Node selection (LLM-driven)

ARI deliberately does *not* reduce node ranking to a closed-form scalar utility. Instead, both `select_next_node` ("which pending node should the next agent step run?") and `select_best_to_expand` ("which completed node is most worth expanding?") hand a list of candidate descriptions to the LLM and parse back a single 0-based index — each call selects exactly one node. The candidate description for each node n is a one-liner of the form

```
[i] id=..., depth=..., label=...,
has_real_data=..., metrics={...},
summary=...
```

The run-selection variant (`select_next_node`) additionally appends `diversity_bonus=+0.05` to the line for any node that currently earns the bonus; `select_best_to_expand` omits it. The prompts that consume the list are reproduced verbatim as section B.3 (selection) and section B.2 (expansion target). The selection criteria the prompt enumerates are: nodes with `has_real_data=True` and strong metrics are high-value; metrics are weighed holistically (multi-objective); deeper nodes with excellent results

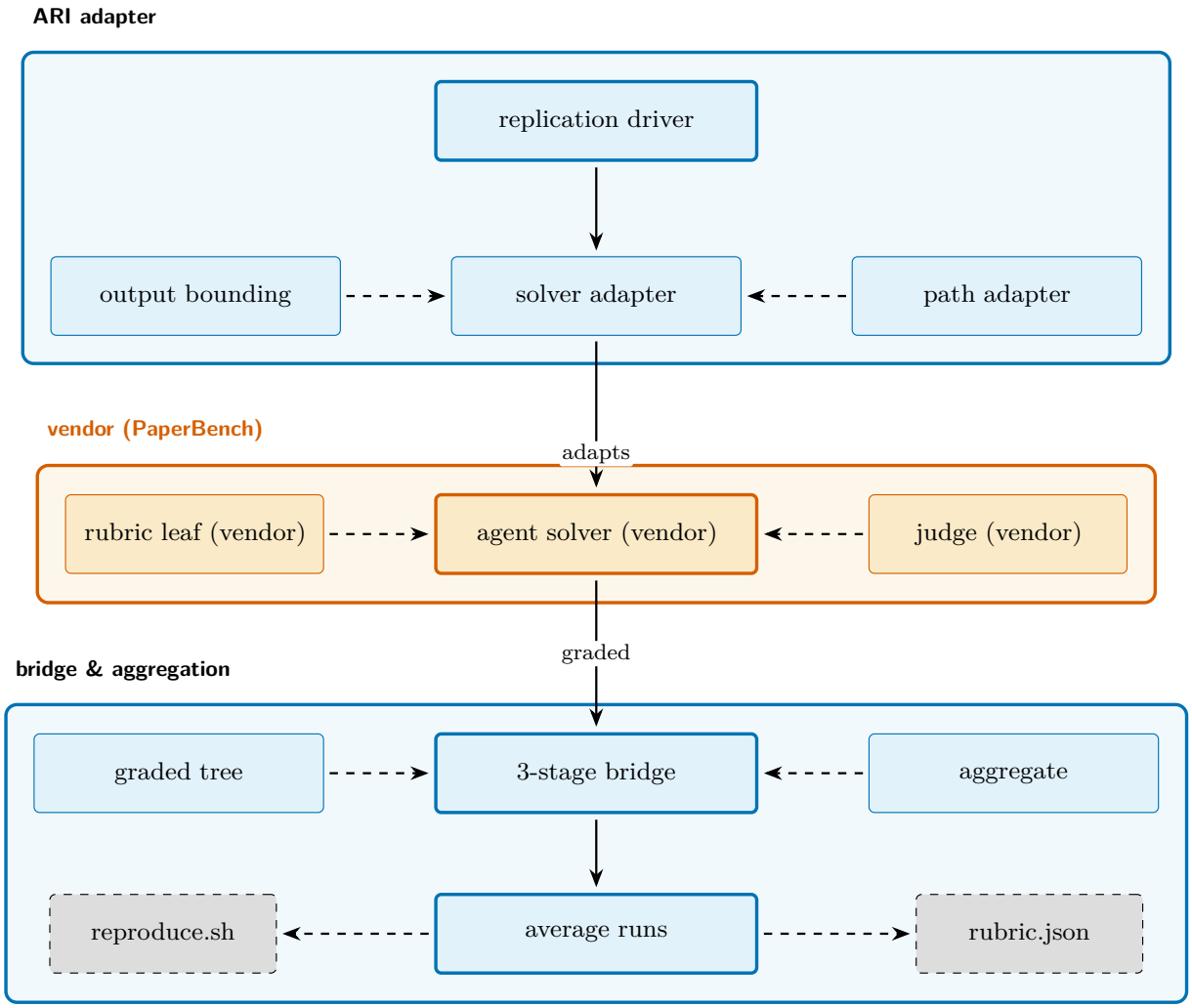

Figure 5: The `paper-re` skill's components in three layers. The top row is ARI's adapter layer (replication driver, solver adapter, output bounding, path adapter); the middle row is the vendored PaperBench package (agent solver, rubric leaves, judge); the bottom row is the bridge that collapses a per-submission graded tree to a single `rubric.json` score plus an ARI-side `reproduce.sh`. Detail in section 4.5.

are worth continuing; the `diversity_bonus` is treated as a soft tiebreaker only. The `label` field matches the information shown to `select_best_to_expand`, and there is no spurious "low retry" criterion.

Diversity bonus

The diversity bonus `BFTS.diversity_bonus` tracks recent labels in `_recent_label_history` (a sliding window populated by `record_run`). For a node whose label is *under*- represented relative to the most common label in that window, the function returns

$$\mathrm{div}(n) = \begin{cases} +0.05 & \mathrm{cnt}(n) \le \frac{1}{2}\,\mathrm{cnt}_{\max}, \\ 0 & \text{otherwise}, \end{cases} \tag{4}$$

where $\mathrm{cnt}(n)$ is the number of times $\mathrm{label}(n)$ occurs in the sliding window and $\mathrm{cnt}_{\max} = \max_{\ell} \mathrm{cnt}(\ell)$. The constant 0.05 is small by design: scientific score still dominates ranking, and the bonus only breaks near-ties.

LLM fallback

If the LLM returns an unparseable index, `select_next_node` falls back to a deterministic ranking via the `_select_fallback` helper (which ranks candidates by the `_fallback_score` expression). The default scoring expression is

$$\mathrm{fb}(n) \;=\; r(n) + \mathrm{div}(n), \tag{5}$$

where $r(n) \equiv$ `_scientific_score`(n) is the node's scalar reward (section A). `_select_fallback` prefers nodes with `has_real_data=True` when any are available, then returns the candidate with the highest $\mathrm{fb}(\cdot)$. This guarantees the loop always makes progress even when the LLM call degrades.

Configurable evaluation layers

The four evaluation surfaces in sections 3.1 and 3.3 are independently configurable; the defaults reproduce the equations above, so an unmodified configuration is a no-op.

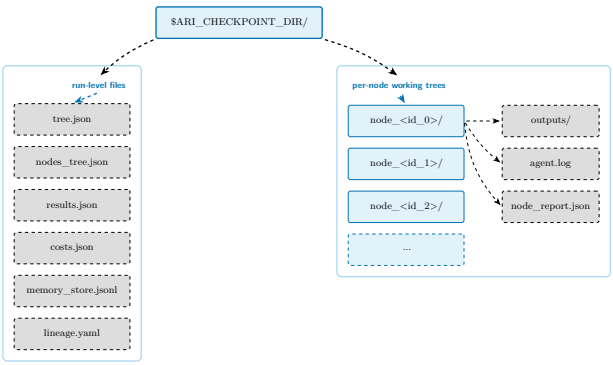

Figure 6: Checkpoint directory layout. Run-level files (left band) are written by the orchestrator; per-node working trees (right band) hold the artefacts produced inside one BFTS expansion. Reproducing a run is exactly "re-execute against this directory", and the layout guarantees that no other state is needed.

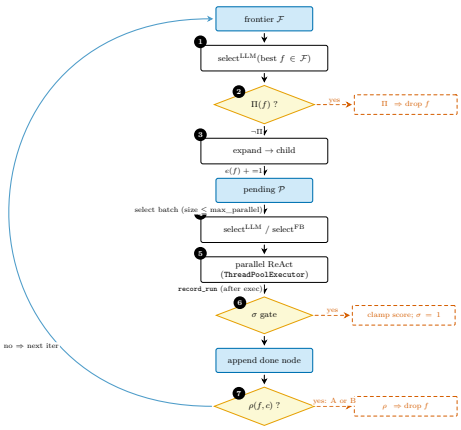

Figure 7: State-machine view of one outer BFTS iteration as a single top-to-bottom flowchart. Steps 1–3 are the inner expansion sub-loop; steps 4–7 are the outer parallel-run loop. Yellow diamonds are the predicate guards (Π, σ, ρ); red dashed boxes on the right are the side-effects each predicate triggers (frontier drop / score clamp / retire). Auxiliary state not drawn here: the per-parent expansion counter $e(\cdot)$ is bumped at step 3 and consulted by ρ at step 7, and the label history H (window W) is appended at step 5 via `record_run` and consulted at step 4 to compute the diversity bonus $\text{div}(\cdot)$. Compare with fig. 4, which shows the same loop as a finer-grained control-flow spine with the LLM prompts drawn in.

Composite formula. Selected by `evaluator.composite`. The reduction $\{a_k\} \to$ `_scientific_score` is one of: weighted harmonic mean (default), arithmetic mean, bottleneck `weighted_min`, or weighted geometric mean. The harmonic and geometric forms apply a floor $\epsilon = 0.01$ to a zero-valued axis so the denominator stays finite.

Frontier-score strategy. Selected by `bfts.frontier_score`. eq. (5) is the `scientific_plus_diversity` case. The alternatives are `scientific_only` (drops div); `depth_penalized`, adding $-\lambda\,\text{depth}(n)$ on top of $\text{div}(n)$ with $\lambda =$`depth_penalty_lambda`; and `ucb_like`, adding $c_{\text{ucb}}\sqrt{\log N/(e(n)+1)}$ on top of $\text{div}(n)$, where $c_{\text{ucb}} =$`ucb_c` and $N = \sum_{n'} e(n') + |\mathcal{F}|$.

Axis set. Selected by `evaluator.axis_mode`. `dynamic` (default) builds the axis set from the generic 5-axis floor plus the active rubric and `idea.json` plan keywords. `legacy` pins to the 5 canonical axes. `custom` feeds an explicit $\{(\text{name}, \text{description}, w)\}$ list verbatim through to the judge LLM.

Selection prompts. Set via `bfts.select_prompt` and `bfts.expand_select_prompt`: a pair of `FilesystemPromptLoader` keys lets the user swap in alternative templates for the two select$^{\text{LLM}}$ calls in algorithm 1 without editing source code. The selection template must accept the placeholders `experiment_goal`, `memory_context`, and `candidates`; the expansion-target template accepts `experiment_goal` and `candidates`.

3.4 Expansion (one child per call)

The `expand` function generates *at most one* new child per invocation (no pre-batching). Callers wanting more children for the same parent call `expand` repeatedly, threading the previously created children through the `existing_children` argument so the LLM can avoid proposing duplicate directions.

The label of each new child is decided by the LLM rather than by a hardcoded template; the expansion prompt, reproduced verbatim as section B.1, is fed:

- the parent's status (`succeeded` / `failed/no-real-data`) and `_scientific_score`;
- the parent's structured `node_report` (`delta_vs_parent` / `concerns` / `hints`, produced by the node-report builder) when present, so the planner can target specific weaknesses;
- sibling scores at the same depth, and ancestor scores from root to parent;
- tree-wide diversity metrics (unique labels seen so far, depth distribution); and
- the labels of children already spawned at this parent, so duplicates are not proposed.

A child is created in state open, handed to the agent loop for execution (section 3.8), and transitions to done when all axes are populated or to failed if the agent loop raises.

3.5 Pruning and termination

`should_prune` implements the pruning predicate Π (eq. (1)): total-node cap, depth cap, and the sterile flag. The depth check activates the `max_depth` config field. The frontier retire predicate ρ (eq. (3)) is applied after each node completes: Rule A retires a parent

Algorithm 1 BFTS run-loop (one outer iteration).

Input: state $S = (\mathcal{F}, \mathcal{P}, e, H)$; config $(B_N, B_D, E_{\max})$ and worker cap `max_parallel` $= \min(\texttt{max\_parallel\_nodes}, 4)$
Output: updated state S'

▷ — inner expansion sub-loop —
while $\mathcal{F} \neq \emptyset \wedge |\mathcal{P}| < \texttt{max\_parallel} \wedge |\mathcal{N}| < B_N$ **do**
  $f \leftarrow \text{select}^{\text{LLM}}(\mathcal{F})$ ▷ `select_best_to_expand`
  if $\Pi(f; |\mathcal{N}|)$ **then**
    $\mathcal{F} \leftarrow \mathcal{F} \setminus \{f\}$; **continue**
  end if
  $c \leftarrow \texttt{expand}(f, \texttt{depth\_note}, \texttt{budget\_note})$
  $\mathcal{P} \leftarrow \mathcal{P} \cup \{c\}$; $e(f) \mathrel{+}= 1$
end while
▷ — outer parallel-execution batch —
$B \leftarrow \emptyset$
while $|B| < \min(\texttt{max\_parallel}, |\mathcal{P}|)$ **do**
  $n \leftarrow \text{select}^{\text{LLM}}(\mathcal{P})$ ▷ `select_next_node`: one node
  $\mathcal{P} \leftarrow \mathcal{P} \setminus \{n\}$; $B \leftarrow B \cup \{n\}$
end while
for all $n \in B$ **in parallel do**
  $n' \leftarrow \text{ReAct}(n)$ ▷ `AgentLoop.run`, may fail
  $H \leftarrow (H \parallel \text{label}(n'))[-W:]$ ▷ `record_run`
  $\mathcal{F} \leftarrow \mathcal{F} \cup \{n'\}$
  for all $p \in \mathcal{F} : p = \text{pa}(n')$ **do** ▷ Rule A
    if $r(n') > r(p)$ **then**
      $\mathcal{F} \leftarrow \mathcal{F} \setminus \{p\}$
    end if
  end for
  for all $p \in \mathcal{F}$ **do** ▷ Rule B
    if $e(p) \geq E_{\max}$ **then**
      $\mathcal{F} \leftarrow \mathcal{F} \setminus \{p\}$
    end if
  end for
end for
return $S' = (\mathcal{F}, \mathcal{P}, e, H)$

f once a child's reward $r(\cdot)$ surpasses it, and Rule B retires f once it has been expanded $E_{\max}$ times. ρ does not delete the node — its history stays on $\mathcal{T}$ — but the frontier pool shrinks so the search spreads rather than mining the same parent indefinitely.

The loop terminates when any of:

- the global `max_total_nodes` cap is hit;
- *every* frontier node fails Π (depth cap, sterile, or budget exhausted) and the pending pool is empty;
- the per-call step / cost budgets enforced by the cost tracker run out;
- the *stagnation detector* `detect_stagnation` observes that the running max of `_scientific_score` has not improved over a sliding window;
- a `LineageDecision` from the lineage-decision module explicitly halts the branch (see section B.4).

Which of these fires most often is an empirical question that section 5 is meant to answer once a frozen checkpoint is in place; the present report does not claim a distribution.

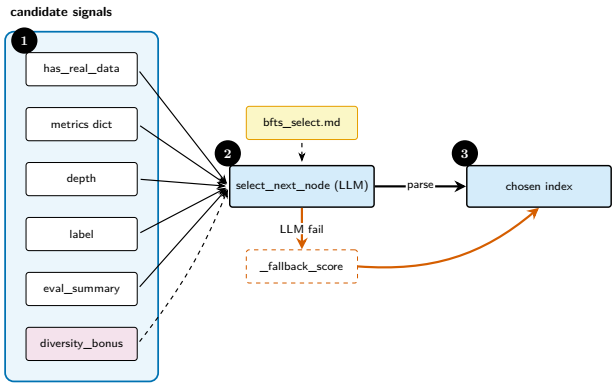

Figure 8: Selection signals fed to the LLM that picks the next node to operate on. Inputs are concatenated into a structured candidate description; the soft `diversity_bonus` is a tiebreaker, not a primary signal.

3.6 Lineage and reproducibility

A lineage is the chain of ancestors of the best node, written into the checkpoint as the `lineage` key in `tree.json`. The `LineageState` dataclass captures the running statistics that BFTS uses to decide whether to continue, and the cross-checkpoint walker `walk_ancestor_ckpts` provides parent pointers across runs. Idea pools are inherited via `get_idea_pool_for_ckpt`, and a ranked root-idea selection is recorded by the `RootChoice` dataclass so the choice is auditable post hoc (prompt: section B.5).

Reproducibility hinges on three invariants. (i) Every randomised operation seeds itself from the node identifier. (ii) Every tool call records its arguments and outputs into the node's working tree, never the parent's — this is enforced by the MCP layer's `cow_node_id` check. (iii) The cost ledger attaches a per-node dollar amount at `CostTracker.init`, so the budget check is deterministic given the same prompts.

Offline by default. Reproducibility also dictates what the search loop is allowed to read. By default a node's agent loop runs *offline*: the web-access skill is phase-gated to the paper-writing and replication stages, so a node cannot consult time-varying live search results during the search itself, and the per-node trajectory therefore does not depend on the state of the web on the day it ran. Literature lookup still happens, but earlier and out of band — a bounded survey performed at idea time, before the search begins. A run that genuinely needs live information can opt in (`bfts.allow_web`) to expose web access to the node agent during exploration; when it does, the run writes a non-reproducible-trajectory marker into its checkpoint, so a later reproduction is never silently treated as exact. The determinism contract stays honest either way: the default run is byte-reproducible, and a run that trades reproducibility for live information records that trade in its own provenance.

3.7 Lineage chaining for computed evidence

Not every claim a run sets out to support is measured fresh. Some evidence is *computed* from measurements that already exist — a parameter fit over earlier probes, a held-out validation of that fit, a model-based selection among configurations. Such claims turned out to be structurally unreachable by independent tree nodes: in a real run, children directed at fitting or validation but expanded from a parent that carried no measurements regressed to re-running single probes, because "compute from the data you already have" was never a visible option — the child had no data, and nothing told the selector which node did.

The mechanism that closes this gap uses only the parent→child working-directory inheritance the tree already has (a child executes inside a copy of its parent's working tree; section 4.2 relies on the same fact), as one hint per side of the expansion. On the parent side, the expansion-target selection prompt (section 3.3) is extended with a coverage hint that names the node whose working tree holds the most *contract-evidence names* — measurement names required by the run's declared metric contract, the set of falsifiable claims the eventual paper must evidence (section 4.8 defines it in full) — so a fit-or-validate child is preferentially expanded from the node that already holds its inputs. On the child side, a child whose inherited directory contains lineage measurement files is told, at start, which files are present and which exact contract names they contain, and is instructed to compute the derived evidence from those files rather than re-run the underlying experiments.

Only names cross node boundaries: file names and measurement names, never values and never a sibling's conclusions. The hints are run-level coordination metadata of the same kind as the contract itself, so the branch isolation of section 3.9 is preserved — a faulty branch's numbers still cannot leak into a sibling's reasoning — while the computed-evidence chain (fit, then validate, then select) can execute along a single lineage instead of collapsing back into repeated probes.

3.8 ReAct driver

Each node's expansion runs a ReAct-style agent loop in the `AgentLoop` class. The maximum step count is `MAX_REACT_STEPS = 80`; it can be overridden per-instance via the `max_react_steps` keyword. A separate guard `MIN_TOOL_CALLS = 2` prevents trivially short trajectories. The agent's system prompt is reproduced verbatim as section B.6.

The set of tools available to the agent is filtered per phase by the tool manager; for example, the exploration phase masks the paper-writing tools so that BFTS expansion cannot short-circuit into a draft. A small set of MCP tools is reserved for `ari-core` itself (memory CoW operations) and is hidden from the LLM, ensuring that the model cannot set an arbitrary

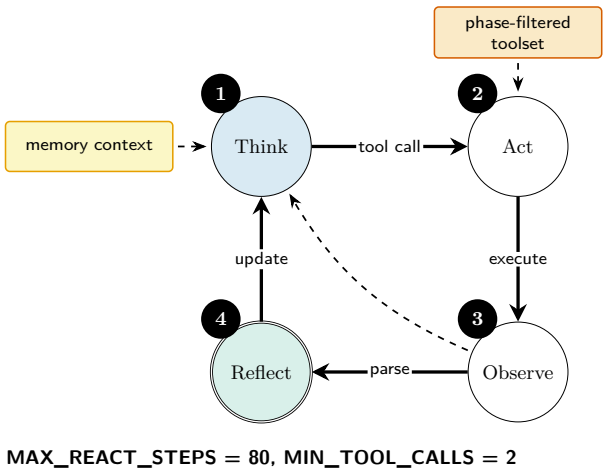

Figure 9: The ReAct loop. Solid edges are the main cycle; the dashed back edge represents an early-exit when the observation already closes the open question.

node id and bypass the memory skill's copy-on-write check.

3.9 Research memory

Nodes do not run in isolation: each one inherits what its ancestors established and, in turn, leaves a record for its descendants and for the paper writer. ARI keeps this record in a per-run *research memory* governed by two requirements a free-form log does not meet. First, *branch isolation*: a node may read only the memory of its own ancestors, never that of a sibling or an unrelated checkpoint, so two parallel leads cannot contaminate each other — the same ancestor-scope predicate that defines a lineage in section 3.6. Second, *groundability*: because the memory ultimately feeds a paper, an entry must be able to point at the evidence behind a result rather than merely restate the result, so a claim can be traced back to the artefact that supports it.

Memory as an index over evidence

The single source of truth for what a node produced is its structured self-report — the `node_report.json` written when the node completes (section 4.2 details its fields). A memory entry does not copy those fields. It carries a short searchable text, a *kind*, a pointer back to the originating node report, and content-hash references to the specific artefacts — logs, source files, output files — a result rests on. A memory entry is thus an *index into evidence*, not the evidence itself: the artefacts it cites can be re-hashed at any time to confirm they still exist and still match, an integrity check that classifies every reference as verified, missing, or mismatched. The *kind* is drawn from a fixed vocabulary — observation, experiment result, failure case, procedure, reflection, artefact summary, paper claim, and reproducibility event — so a reader can ask for exactly the records it needs: the failures along a branch, say, or only the results that are backed by

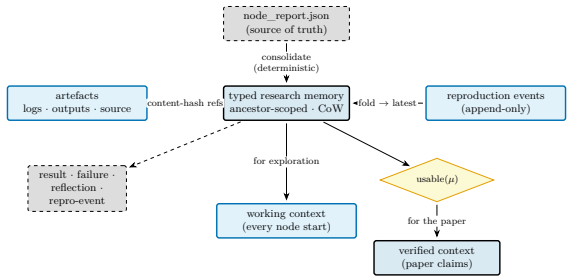

Figure 10: Verifiable research memory. A deterministic consolidation of each node's `node_report.json` — the source of truth — populates a *typed* store that *indexes* artefacts (logs, outputs, source) by content hash. Reproduction events are appended and folded down to a latest status, and an admissibility predicate usable(μ) (eq. (6)) gates which records reach the verified context. The same store yields two readers: the *working context* injected at the next node's start (the inheritance path of exploration) and the *verified context* that grounds the paper's quantitative claims (section 4.2). Ungrounded reflection may steer exploration but never the paper body.

artefacts.

Consolidation at node end

When a node completes, a deterministic *consolidation* step reads its node report and emits typed entries: an experiment-result entry, with provenance references to the artefacts it measured, for a node that produced measurements; a failure-case entry for one that failed. The step invokes no language model, so the same node report always consolidates to the same entries — consolidation is a pure function of recorded state, and re-running a checkpoint reproduces the memory it produced. It is a loop hook, not an agent action: the search agent is never asked to "save to memory", and in practice the agent does not reach for the recall tools on its own. The memory reads the loop depends on are performed by the loop itself, deterministically — which is what lets the memory carry provenance without putting a non-deterministic retrieval on the critical path (section 7.1).

Two readers: working and verified context

The store feeds two consumers (fig. 10), and the distinction between them is the heart of the design. At the *start* of every node the loop injects a *working context*: the experiment's fixed core — goal, target metric, hardware — together with the conclusions its ancestors recorded. A child therefore begins from what its lineage has already established, deterministically and scoped to its own branch, rather than from an aggregate text dump. At *paper* time the pipeline builds a *verified context* from the best node's lineage: it folds each result's reproducibility events down to a latest status and ranks entries so that reproduced results outrank merely-artefact-grounded ones, which in turn outrank ungrounded reflection. An entry μ is *admissible* as a quantitative paper claim only when it

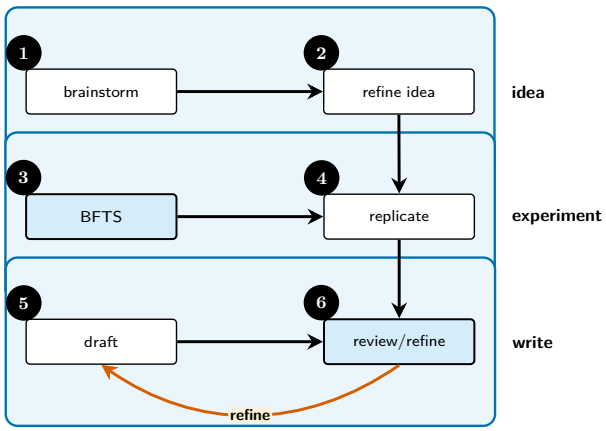

Figure 11: Paper-generation swim lane: idea, experiment, write. Each lane corresponds to a phase; the back edge from review to draft is the only non-monotonic transition (section 4.4).

is artefact-grounded and not contradicted by a reproduction attempt:

$$\text{usable}(\mu) \;=\; \text{grounded}(\mu) \;\wedge\; \big(\text{repro}(\mu) \neq \text{failed}\big). \tag{6}$$

Ungrounded reflection can still steer the search through the working context, but eq. (6) holds it out of the paper body: the writer's quantitative claims rest only on admissible, reproduced-first entries. This is the memory-side counterpart of the evidence assembler (section 4.2); where the assembler decides which artefact *bytes* the writer may read, the verified context decides which *results* are allowed to become claims.

Reproducibility as an appended event

Because past entries are byte-stable — a node writes only under its own identifier and existing entries are never edited in place — a result's reproducibility status is not patched when the result is later re-run. Instead the re-run *appends* a reproducibility-event entry, and any reader folds the events for a target down to their latest state. A result that fails to reproduce is thereby demoted out of the claim set of eq. (6) — and can be reported honestly as a limitation — without rewriting the history that recorded it as promising in the first place.

4 Paper Generation Pipeline

4.1 Pipeline overview

Once exploration terminates, the surviving lineage is handed to the paper-generation pipeline, the second tree of activity in ARI (fig. 11). A *write* phase compiles the lineage into a draft, and a *review* phase scores that draft against a rubric; the two are coupled by a refine loop (section 4.4). The pipeline assembles, from the per-node artefacts left by exploration, the evidence the writer is allowed to draw on (section 4.2), and persists three outputs: the LaTeX draft, the per-leaf review with its rationale, and the rubric instance

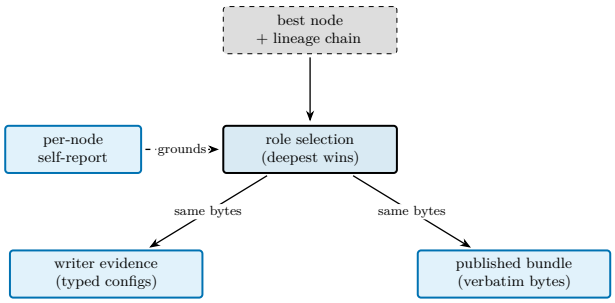

Figure 12: Evidence assembly. The deterministic assembler turns the best node's lineage into a *single* selection that feeds *two* consumers — the writer's lineage evidence and the published artefact bundle — from the same selected bytes. The per-node self-report (content hashes, success self-assessment, literal build/run commands, captured hardware) grounds the selection, so each contributing file is chosen by recorded metadata; on a path collision the deepest occurrence wins, and a chain deletion removes the file.

used to score the run (kept because the rubric may be generated per paper rather than fixed in advance).

4.2 Evidence assembly

Between the two trees sits a deterministic *evidence assembler* (fig. 12). It decides which of the lineage's per-node artefacts the writer may draw on and reshapes them into a single science-facing record. The design keeps this seam auditable — which node a claim traces to is a function of recorded metadata, not of a language model's judgement — so that the guardrails of section 4.7 have something concrete to enforce.

Selection is role-specific: one node may contribute differently to three consumers, and a single predicate family decides each. A node enters the *synthesis* set if it carries real measurements (or the evaluator certified its run a success); it enters the *code* set if it introduced or modified a source file relative to its parent; and it enters the *narrative* set if its step succeeded. A node that merely explored a dead end is admitted to none. Each criterion is a pure predicate over recorded metadata, so the same inputs always select the same contributing set.

Source files are gathered by overlaying, in depth order, the files each contributing node touched along the ancestor chain $\mathrm{lin}(n^\star)$ of the best node $n^\star$. Because a child node executes inside a copy of its parent's working tree, one relative path may occur at several depths; the deepest occurrence wins, and a deletion along the chain removes the file — so the reconstructed set matches the best node's actual working tree. This selection then feeds *two* consumers from the same bytes: the writer's lineage evidence and the published artefact bundle, so the lineage code the paper describes is the code that is released. Selection is chain-only — a node off the best lineage is not published even when its measurements are reported by the synthesis set, and is logged for provenance — and, as a fidelity

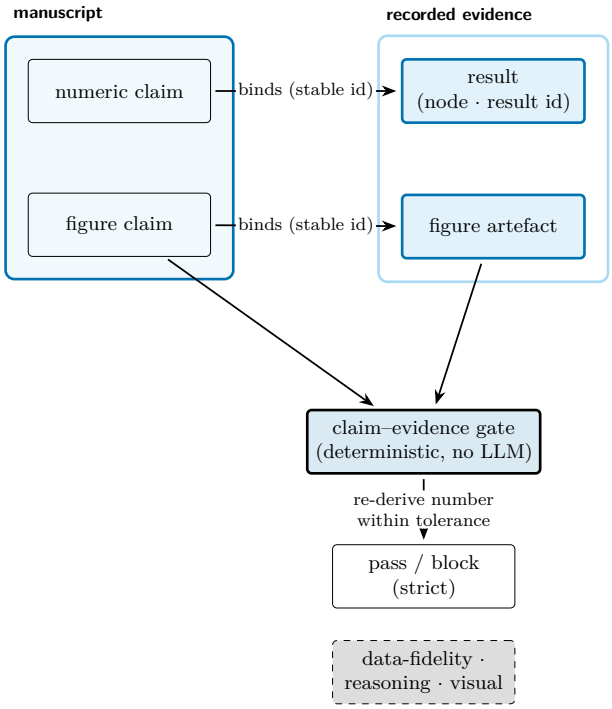

Figure 13: An execution-grounded realisation of the Story2Proposal claim–evidence contract. Manuscript claims — numeric assertions and figure references — are bound by stable node, result, and figure identifiers to the recorded evidence produced by executed experiments. A deterministic claim–evidence gate (no language model) re-derives each reported number from that evidence within a tolerance; this data-fidelity axis blocks in strict mode, while the reasoning and visual-coherence axes are non-blocking.

aid for pseudocode, the writer is additionally shown the raw source of the top-scoring nodes, which is not itself the published set.

The assembler reads code, parameters, and measurements *directly* from each node's working tree rather than from a summary. Where an experiment declared a typed result, the record separates *input parameters* (configuration knobs — problem size, thread count, seeds) from *measured outputs* (throughput, accuracy, latency) and from derived quantities (efficiencies, model-predicted ceilings). The separation is not cosmetic: per-key extrema reported across configurations are reduced deterministically and *only* over measured keys,

$$\mathrm{best}(m) \;=\; \operatorname*{red}_{c \in \mathcal{E}} c[m], \qquad m \notin \mathcal{I}, \tag{7}$$

where red is a maximum or a minimum according to the metric's declared direction, $\mathcal{E}$ is the set of contributing configurations, and $\mathcal{I}$ is the set of declared input keys held out of every reduction. Holding the inputs out is what stops a large input magnitude — a problem size of several million nonzeros, say — from being read as a headline result. The language model is confined to the one task that genuinely needs it: turning verbatim source and logs into prose and pseudocode and narrating methodology. It never supplies the numbers.

Algorithm 2 Evidence assembly (exploration → writer input).

Input: lineage nodes $\mathcal{N}$ with per-node reports; metric-direction map
Output: science record $\mathcal{E}$: typed configurations, per-key extrema, methodology narrative

$n^\star \leftarrow \arg\max_{n \in \mathcal{N}} r(n)$ ▷ ties: validated, then deeper
$L \leftarrow \mathrm{lin}(n^\star)$ ▷ root-to-best ancestor chain
$\mathcal{C} \leftarrow \{\, n \in L : \textsc{Contributes}(n) \,\}$ ▷ role predicate (code)
$F \leftarrow \emptyset$ ▷ relative path ↦ (node, depth)
for all $n \in L$ in depth order **do**
  for all path p added/modified by $n \in \mathcal{C}$ **do**
    if $p \notin F \vee \mathrm{depth}(n) \geq \mathrm{depth}(F[p])$ **then**
      $F[p] \leftarrow (n, \mathrm{depth}(n))$ ▷ deepest wins
    end if
  end for
  for all path p deleted by n **do**
    remove p from F ▷ deletion overlays it away
  end for
end for
load the bytes of every $p \in F$ verbatim, under a size budget
for all measured key $m \notin \mathcal{I}$ **do**
  $\mathrm{best}(m) \leftarrow \mathrm{red}_c\, c[m]$ ▷ eq. (7); deterministic
end for
$\textsc{Narrate}(F)$ ▷ LLM: verbatim source/logs → prose, pseudocode
return $\mathcal{E}$

Grounding all of this is a per-node structured self-report written when a node completes (`node_report.json` in the checkpoint). It records, by content hash, which files the node added or modified relative to its parent; the node's own success self-assessment together with the concerns the evaluator flagged; the literal build and run commands; and the hardware the measurement ran on. The hashes make the source selection reproducible and let the released bundle carry a reproduction script built from the recorded commands; the captured hardware lets the paper's environment section be written from fact rather than left as "not recorded".

4.3 Rubric formalism

We model the *rubric* R as a finite tree whose leaves carry tuples $(c_i, w_i, \mathrm{judge}_i)$. Each leaf i has a category descriptor c_i, a non-negative weight w_i with $\sum_i w_i = 1$, and a *judge function* $\mathrm{judge}_i : P \to [0, 1]$ that returns a graded score. The paper score is the convex combination

$$\mathrm{score}(P) \;=\; \sum_i w_i \,\mathrm{judge}_i(P). \tag{8}$$

This tree-structured rubric is shared with PaperBench, which ARI reuses through a vendored package (section 4.5) so that its grade and aggregation semantics track upstream verbatim. Two judge families coexist: an LLM-backed judge for qualitative leaves, and a deterministic judge for leaves with a binary or numeric verifier (e.g. "is the cost reported in absolute dollars?"). Both satisfy the same `Evaluator` interface, so the search engine (section 3) and the paper pipeline reduce axes to a scalar through one seam.

A rubric is generated in one of two *modes*. In *agent-benchmark* mode the tree decomposes the paper by scientific contribution and each leaf asks whether a candidate submission's executed output reproduces the paper — the original PaperBench framing. In *paper-audit* mode the tree's top-level children are a fixed set of reproducibility axes and each leaf grades the descriptive completeness of the paper text itself — a framing inversion aimed at HPC reproducibility audits, where the question is not "can an agent reproduce this paper?" but "does this paper describe enough to be reproducible?". Venue knowledge (which axes, how they are phrased) is supplied by a per-venue template rather than baked into the engine, so ARI core stays domain-agnostic (P4) and a new venue is a data change, not a code change.

Lifting the paper-audit score out of a degenerate regime — where an empty submission drives every submission-oriented leaf to "no" — required rephrasing the leaf questions to interrogate the *paper* rather than a (missing) submission, feeding the judge the paper's figures alongside its text so vision-capable models can use them, and admitting optional artifact-description and artifact-evaluation appendices as extra input. These adjustments live entirely in ARI's wrapper layer; the vendored judge is untouched, which keeps scores comparable across upstream versions. We deliberately do *not* synthesise a reproduction log from the paper's own reported numbers: doing so would short-circuit the executed-submission stage and inflate the score in a way not comparable to leaderboard runs.

4.4 Review/refine loop

The refine cycle iteratively rewrites P_t to P_{t+1} under the review feedback $J(P_t)$:

$$P_{t+1} \;=\; \mathrm{Refine}\big(P_t, J(P_t)\big). \tag{9}$$

The cycle stops when the reviewer accepts the draft — modelled as the *convergence criterion* $\mathrm{score}(P_{t+1}) - \mathrm{score}(P_t) < \epsilon$ — or when a small per-section revision cap is reached. Which branch fires more often is an empirical question section 5 is meant to answer once a frozen checkpoint exists; we make no quantitative claim here.

The refine step is intentionally non-monotonic per axis: fixing a clarity issue may slightly degrade a numeric axis. This is the source of the back edge in fig. 11, and it is why the convergence criterion is defined on the aggregate score rather than per-axis — together with a per-axis floor (section 4.7) that prevents the aggregate from hiding a collapse on any single axis.

4.5 PaperBench integration

The replication path is *not* a re-implementation of PaperBench: the upstream package is vendored unchanged so that ARI tracks its task and rubric semantics verbatim, and ARI contributes only a thin wrapper around it. The wrapper exposes the protocol-faithful loop as three composable stages with a shared vocabulary (fig. 14):

1. *rollout* — a replication agent works from the paper towards a submission (code, and optionally a reproduction script);
2. *reproduce* — the submission is executed in an isolated container, on the local machine or dispatched to a cluster;
3. *grade* — the vendored judge scores the submission against the rubric tree.

Two design choices keep this faithful to the benchmark. First, grading is left to the vendored judge; ARI's adapters only reshape its output and aggregate repeated runs into a single score, so upgrading PaperBench is a vendor swap with no change to how leaf grades are combined — the same envelope feeds eq. (8), with $\text{judge}_i \in \{0, 1\}$ for binary leaves. Second, the wrapper *fails loud*: when a benchmark-fair run cannot be served (no container runtime, no scheduler, no GPU resource registration) it raises rather than silently degrading to a weaker host, because a silent downgrade would make the resulting score incomparable to a leaderboard entry. Legacy silent-fallback behaviour is available only behind an explicit opt-in.

A second concern is keeping the replication agent honest about what the host actually provides. Left to the upstream prompt, an agent tends to scaffold a reproduction script that calls binaries the host lacks, and to default to Python regardless of the paper's real language. ARI counters both by priming the agent to probe the environment before it scaffolds, and by injecting a per-host notes block — generated from a live probe of the available toolchain, GPUs, and network reachability — so that what the agent reads matches what the host can do.

4.6 Domain breadth: the execution-profile contract

PaperBench upstream assumes a single container with one GPU. To cover papers whose methodology is fundamentally parallel — multi-rank MPI kernels, multi-node training, cluster-specific module environments — ARI extends the rubric with an optional *execution profile* (fig. 15). The profile is domain-agnostic: it names a *parallel-execution shape* (single-CPU, single- or multi-GPU, MPI, MPI+GPU) together with the paper's scale envelope and any scheduler hints, rather than anything about the paper's scientific field. When a paper is single-machine the profile is omitted and the pipeline degenerates to the original single-node invocation: the contract is additive, not breaking.

The profile feeds two consumers (fig. 15). It is appended to the agent's prompt as a compact "compute-node execution conventions" block — shared-filesystem discipline, a preference for the scheduler's launcher over a bare MPI launcher, environment activation, and wall-clock wrapping — and it parameterises the cluster dispatch (node, GPU, memory, and placement requests). Because clusters disagree on which scheduler flags they accept, the dispatcher probes the local scheduler at run time and drops flags it does not support, so the same rubric dispatches faithfully on a multi-GPU cluster, on a sandbox partition without GPU registration, or on a single workstation. A regression guard enforces the inverse: ARI running inside an unrelated cluster allocation must never leak cluster- or MPI-specific prose into the prompt for a non-HPC paper. Only this wrapper layer changes; the vendored benchmark and ARI core are untouched across the extension.

4.7 Failure modes and guardrails

Four failure modes are common enough to be named, each with a structural — not merely advisory — mitigation:

- **LLM-only support**: the paper asserts a claim that no node in the lineage substantiates. The mitigation constrains the writer to a pre-selected set of lineage artefacts, so every numerical or causal claim is traceable to a node that produced it; the draft cannot cite evidence that was not generated. The companion constraint is on the memory side: the verified context (section 3.9) admits a result as a quantitative claim only when it is artefact-grounded and has not failed a reproduction attempt (eq. (6)), so a reproduced result is preferred and an unsupported reflection cannot become a headline number.
- **Citation hallucination**: the paper cites a non-existent work. The mitigation is structural — the writer may not invent citation keys, and every bibliography entry is verified against an external index before it is admitted (section 6).
- **Refine drift**: the aggregate score climbs while the paper loses coherence. The mitigation is a per-axis floor: if any axis falls below a threshold the refine is aborted, so a gain on one axis cannot mask a collapse on another.
- **Number drift**: a reported quantity does not match the recorded result it summarises. Because every numeric claim names the node and measurement it rests on (section 4.2), a deterministic claim–evidence gate re-derives each from the recorded results and compares it within a tolerance (fig. 13); a value that fails to match the recomputed result is flagged. By default the gate reports such a mismatch without blocking; under a stricter audit policy an unresolved mismatch

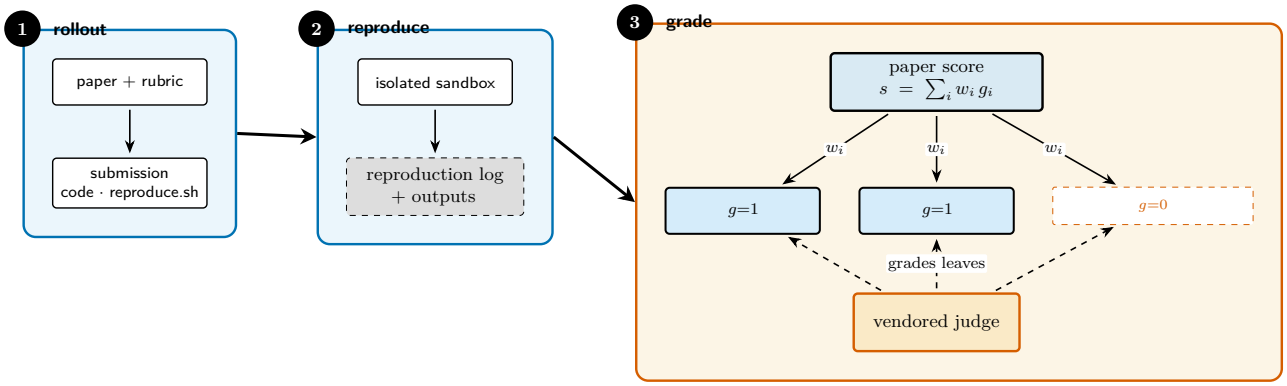

Figure 14: The PaperBench replication path as three composable stages. The agent *rollout* turns the paper and its rubric into a submission (code and a reproduction script); *reproduce* executes that submission in an isolated sandbox, capturing the reproduction log and its outputs; *grade* scores the submission against the *weighted rubric tree*, where the vendored judge assigns each leaf a grade $g \in \{0, 1\}$ and the paper score is the weight-aggregated roll-up $s = \sum_i w_i g_i$ (eq. (8)).

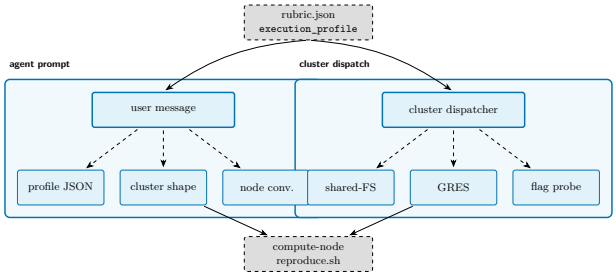

Figure 15: The execution-profile contract threads from a single rubric artefact into two consumers: the replication agent's prompt (left) and the cluster dispatcher (right). Runtime probes adapt the dispatched resource request to the local scheduler, so one rubric runs faithfully on heterogeneous clusters without being modified.

in the final check rejects the draft rather than publishing it.

The first guard also bounds context: the writer is given the selected lineage artefacts and the top-scoring nodes' source, under a size budget, so the draft stays within the model's context window even for long runs.

4.8 Metric-contract and gate hardening

The number-drift guard above presupposes a stable vocabulary binding claims to evidence. ARI makes that vocabulary explicit as a *metric contract*: a run-level declaration of the falsifiable claims the eventual paper must support, each naming the exact measurement names that count as evidence for it. Nodes emit measurements under those names during exploration — the same names also steer expansion (section 3.7) — and the deterministic gate of fig. 13 matches claims to evidence by exact name. Experience with real runs hardened four points of this seam, each a structural rule rather than a prompt fix:

- **The contract is minted once.** Once a claims-bearing contract has been persisted, every later call returns the persisted contract verbatim instead of regenerating it. A language model's naming is not referentially stable — asked twice, it names the same quantity differently — and in a real run a mid-run regeneration silently renamed the evidence vocabulary, hiding measurements that sibling nodes had already emitted from the exact-match gate.

- **Parsing meets the writer where it is.** The gate's numeric verification parses scientific notation and treats math delimiters as unit boundaries when extracting a value–unit pair, and writer declaration quirks that recur in practice — synonymous formula names for the same derivation, claim identifiers written with a label prefix, one in-text anchor identifier (the marker that binds a reported number to its recorded result) shared by several mentions — are normalised at parse time rather than reported as mismatches, so the gate's verdicts measure the paper's numbers, not its surface syntax.

- **Review findings reach the refiner whole.** Every warning raised by the semantic review — a complementary language-model pass that flags overclaims and unsupported causal language — is forwarded to the refine step (section 4.4), not a truncated subset, and the post-refine resolved count is a raw delta between the two passes, so a refine that introduces new findings shows up as a regression rather than being clamped away.

- **Blocking stays reserved for objective falsehoods.** A reported number that contradicts recorded evidence blocks the final check; the subjective findings of the semantic review are advisory by design, steering the refiner without ever gating publication on a matter of judgement.

5 Preliminary Observations

The observations below are what the running system establishes at this stage. They are preliminary and, for the end-to-end reproduction case, single-run rather than a controlled benchmark; a panel-scale quantitative study is left to future work. Quantitative figures elsewhere in this report are illustrative, generated from fixed-seed sample data so that the document build is reproducible.

Three properties hold directly by construction of the system. Cost accounting is exact: a dollar cost is attached to every model invocation and accumulated, per run, into the checkpoint's results record. The exploration loop terminates through the stagnation detector (section 3.5) rather than only the step or cost budgets ($T_{\max}$ / $C_{\max}$). And the execution-profile pathway behaves as specified (section 4.6): rubric generation emits a multi-rank (MPI) execution profile, with the paper's reported scale envelope, for an HPC paper and omits it for a single-machine paper, while a multi-flag cluster dispatch is accepted by the local scheduler once the runtime probes drop the flags it does not support.

End to end, the v0.8.0 pipeline was exercised on a lossy-compression target, a GPU error-bounded scientific compressor. The degree of reproduction achieved on this single target was partial but substantive. The agent fetched a real slice of the paper's cited simulation dataset rather than substituting synthetic data; compiled native CUDA interpolation kernels and ran them on the GPU; implemented Lorenzo and multi-level interpolation predictors with per-level autotuning; and swept a range of error bounds, recovering reconstruction qualities from roughly 50 to 90 dB PSNR with the corresponding compression ratios. Against the fine-grained rubric the run cleared on the order of a tenth of the weighted leaves, for two reasons rooted in the agent's behaviour: the graded metric came from a CPU predictor — the GPU kernels were built and executed, but the agent omitted their predicted quality from its reported results — and the agent terminated voluntarily after a small fraction of its time budget, leaving the remaining datasets and the proprietary lossless stage unattempted.

On the generation side, one end-to-end result is already demonstrated and inspectable: the released sample paper, an eight-page study produced autonomously by a real run. It studies compressed sparse row (CSR) sparse-dense matrix multiplication (SpMM), selecting a store policy across right-hand-side (RHS) widths under the guidance of a performance model (the paper's "loopline" model) fitted from the run's own measurements. In that run, all twelve declared falsifiable claims carried machine-verified experiment evidence — the final claim–evidence gate (section 4.8) reported zero errors — and the computed-evidence chain behind the selection (parameter fitting, then held-out validation, then model-based selection) executed through lineage-chained nodes (section 3.7) rather than collapsing into repeated probes. Correctness was validated against an independent dense single-precision (FP32) reference, with a maximum absolute error of 6.79×10^{-6}, and the post-refine semantic review reports zero remaining overclaims (five detected, five resolved).

We read this result narrowly. Its value is the *verified* property — every quantitative claim in the released paper is bound to a recorded artefact and passed the deterministic gate — not any headline number the paper reports; and the paper itself hedges explicitly wherever a statement outruns its artefact grounding. A single verified run demonstrates that the contract, gate, and lineage machinery of sections 3.7 and 4.8 compose end to end; it does not establish a distribution, which is exactly the controlled study deferred below.

A controlled evaluation — median run cost, per-seed exploration curves, and per-category rubric distributions over a frozen checkpoint, together with full-rollout replication scores across a panel of contrasting target papers — is left to future work.

6 Related Work

We position ARI against four threads of prior art.

6.1 Tree search for code

AIDE [2] formalises ML-engineering as a tree search over plan / code / debug states; AlphaCode [3] is the canonical reference for large-scale generate-and-filter on competitive programming; the MLE-bench evaluator [4] is now the standard for measuring ML-engineering agents. ARI inherits the BFTS skeleton from AIDE but adds (i) an LLM-driven candidate selector (section 3.3), which avoids fixing a closed-form scalar utility and lets the selection prompt weigh `has_real_data`, full `metrics`, depth, and a soft `diversity_bonus` jointly, and (ii) the cross-run lineage (section 3.6) which neither AIDE nor MLE-bench formalises.

6.2 Autonomous research agents

The AI Scientist family [5, 6] closes a similar loop end-to-end (idea → experiment → paper) but does not expose the rubric and refine cycle as a first-class object. VirSci [7] simulates several scientist personas and uses their consensus to score outputs; ARI can drive VirSci's original deliberation engine directly and unmodified (fig. 16) — its freshness-based `select_coauthors` team formation and multi-agent `generate_idea` loop — over a live Semantic Scholar snapshot, so that the multi-agent mechanism it builds on is the published one executed against current literature rather than a paraphrase of it.

Agent Laboratory [8] treats the agent as a collaborator rather than a driver. ARI differs in two operational respects: **everything lives in a checkpoint** (P1 /

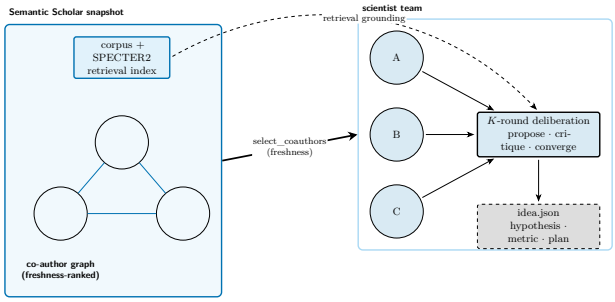

Figure 16: VirSci-style idea generation as ARI drives it. A frozen Semantic Scholar snapshot — a corpus with a SPECTER2 retrieval index and a freshness-ranked co-author graph — is the substrate from which `select_coauthors` forms a heterogeneous scientist team. The team then runs a K-round deliberation grounded by retrieval against the snapshot, converging on `idea.json`.

P4), and the rubric is the contract between explore and write.

6.3 Paper evaluation

PaperBench [9] is the closest analogue to our `paper-re` integration; it pioneered the rubric-as-tree formalism. We adopt the same per-leaf grade and aggregate weight (eq. (8)) but extend it with a per-axis floor used as a refine guard (section 4.7).

Story2Proposal [10] governs structured manuscript generation through a persistent shared contract — section structure, a registry of visual artifacts, and document-wide validation rules — with writer, refiner, and renderer agents coordinating under a generate–evaluate–adapt loop scored on reasoning verification, data fidelity, and visual coherence. It takes the experimental evidence as given and governs how that evidence is written up; it does not execute experiments. ARI integrates that contract-governed-generation principle and its evaluation axes into an execution-grounded pipeline (section 4.2): the contract is realised by extending the structured experiment record with claims and numeric assertions that reference stable node, result, and figure identifiers; data fidelity becomes the deterministic claim–evidence gate that re-derives each reported number from the recorded results; and reasoning and visual coherence become a non-blocking evidence-grounded review that leaves the independent manuscript review intact. Because ARI executes the experiments rather than receiving the evidence as input, it extends this manuscript-level contract to execution-grounded provenance.

6.4 Foundations

The agent loop follows the ReAct pattern [1]; the refine cycle is descended from Self-Refine [11] and Reflexion [12]; and the choice to give the LLM a step-by-step scratchpad in *Think* is descended from Chain-of-Thought [13]. None of these references propose a checkpoint-scoped state; that is ARI's contribution.

An earlier draft cited a separate ML-research-agent benchmark, but we removed that reference once it became clear that none of the candidate sources met our verification rule (section 4.7); we keep the related-work coverage to entries that can be S2-verified.

7 Discussion and Limitations

7.1 Determinism vs novelty

P2, the determinism principle, is the binding constraint of the system. It does not forbid the research memory (section 3.9) from using an embedding model — it constrains *where* that non-determinism is permitted to influence a run. The memory subsystem makes no generative language-model calls, but its archival search ranks by embedding similarity, whose result depends on a remote service and is not byte-reproducible. P2 is honoured by keeping that surface off the reproducible path: the memory reads the search loop actually depends on — the working context injected at each node and the verified context that grounds the paper — are deterministic, ancestor-scoped folds of the per-node reports, not embedding-ranked retrievals. Semantic search stays available as an optional recall aid, but the loop never pulls it, so a re-run reproduces the same trajectory regardless of how the embedding service happens to rank a query.

The cost is a hard line between two kinds of memory read. Anything that must be reproducible is computed by a deterministic function of recorded artefacts; anything that rides on embedding similarity is advisory only and may not, by itself, ground a paper claim. This is not an accident of implementation but the same bargain made everywhere in the system: every reproducibility win comes at the cost of a closed door for non-determinism. We accept the cost because the checkpoint, not the run, is the contract with downstream users.

7.2 Scaling limits

The single-machine assumption underlying the BFTS run-loop is the biggest scaling constraint. A run that wants to expand $b > 10$ in parallel needs a different concurrency layer; today a single `ThreadPoolExecutor` in the BFTS CLI loop is the sole worker pool. A multi-host expansion would require extending the lineage discipline (P4) to handle merges, not just ancestor walks.

A separate constraint is the cost of the LLM grader. Empirically the grader is one of the dominant line items in a run, alongside the explorer; whether it is the second-largest in absolute dollars is something section 5 is meant to settle once a frozen checkpoint exists. Caching grader outputs by (paper-hash, rubric-hash) would help; we have not yet shipped this.

A Notation Table

This appendix collects every symbol used in the body of the report, with its meaning.

Symbol	Meaning
$\mathcal{N}$	set of nodes in the search tree
$\mathcal{T}$	search tree $(\mathcal{N}, E)$
$n \in \mathcal{N}$	individual node
$\mathrm{ch}(n)$	children of n
$\mathrm{pa}(n)$	parent of n
$\mathrm{depth}(n)$	depth of n
$s(n)$	node state $\in \{\text{open}, \text{eval}, \text{done}, \text{failed}\}$
$r(n)$	scalar reward of n (its `_scientific_score` $\in [0, 1]$)
$\mathbf{e}(n)$	per-axis evaluation vector
ϕ	axis $\to$ scalar aggregator
$\mathrm{div}(n)$	diversity bonus (eq. (4))
$\mathrm{fb}(n)$	deterministic fallback ranking (eq. (5))
$T_{\max}, C_{\max}$	step / cost budgets
R	rubric tree
c_i	rubric leaf i's category
w_i	rubric leaf i's weight
judge_i	rubric leaf i's judge
P	paper draft
$\mathrm{score}(P)$	paper aggregate score (eq. (8))
Refine	refine operator (eq. (9))
ckpt	checkpoint directory
$\mathrm{lin}(n)$	lineage chain ending at n

B Runtime LLM Prompts

This appendix lists the LLM prompts that ARI uses at runtime, reproduced verbatim from `ari-core/ari/prompts/` at v0.8.0. Each listing is byte-for-byte identical to the source on disk. Because the exact bytes are what reach the model, the prompts are reproduced as-is in every language version of this report and are not translated.

Orchestrator

B.1 BFTS expansion

```
You are expanding a BFTS research tree node.

{goal_line}Parent node id={parent_id_short}, depth={parent_depth}, status={parent_status}
{depth_note}{budget_note}Parent metrics: {parent_metrics_json}
Parent summary: {parent_summary}
{sci_note}{idea_block}{parent_report_block}
{siblings_block}{ancestors_block}{existing_block}{diversity_block}Propose exactly ONE child research direction
that is the most scientifically valuable next step. For the "label" field, strongly prefer one of the canonical
labels [draft, improve, debug, ablation, validation]; only invent a custom label (e.g. "replication",
"generalization") when none of the five fits meaningfully - the custom string will be preserved as raw_label.
Base your choice on the experimental context above, not on a fixed template.

Label selection guidance:
- 'debug': parent FAILED or has no real data - diagnose and fix it.
- 'improve': parent succeeded and you want to push its metrics higher by tuning parameters, flags, or
algorithms.
- 'ablation': isolate which component drives the parent's gains by removing or varying ONE component. State
explicitly what is removed/varied and what delta vs. the parent metrics you expect.
- 'validation': rigorously verify the parent's claims (different seeds, edge cases, stress tests,
expected-degradation checks).
- 'draft': start a fresh implementation from scratch to introduce a fundamentally NEW perspective (use this
instead of inventing a new label like 'replication' or 'generalization').

Reply ONLY with a JSON array containing exactly one element: [{"label": "<one of:
draft|improve|debug|ablation|validation>", "direction": "..."}]
Example: [{"label": "validation", "direction": "<one-sentence plan>"}]
```

B.2 BFTS expansion-time selection

```
You are selecting which completed research node to expand next in a BFTS tree.

Experiment goal: {experiment_goal}

Completed nodes awaiting expansion:
{candidates}

Select the single most promising node to expand. Prefer nodes with high scientific_score, strong metrics, and
unexplored directions. Avoid nodes that have already been retried many times or are at excessive depth.
Reply with ONLY the index number (0-based).
```

B.3 BFTS selection

```
You are selecting the most promising node to explore next in a research tree.

Experiment goal: {experiment_goal}

Relevant past memories:
{memory_context}

Candidates:
{candidates}

Selection criteria:
- Nodes with has_real_data=True and strong metrics are high-value
- Consider all metrics holistically (multi-objective)
- Deeper nodes with excellent results are worth continuing
- A small diversity_bonus is awarded to underrepresented exploration types; treat it as a soft tiebreaker, not
a primary signal
Reply with ONLY the index number (0-based) of the best node.
```

B.4 Lineage continuation decision

```
You are a research orchestrator. Given the current state of an exploratory research run, decide the next
lineage-level action. Possible actions:
  continue       - current idea is still productive; keep exploring
  switch_to_idea - current idea has stagnated; switch to an alternative
  fanout         - current idea succeeded; explore an alternative in parallel
  terminate      - research thread is exhausted; stop
Choose the LEAST disruptive action that fits the evidence. Prefer continue unless there is a concrete reason to
escalate. When choosing switch_to_idea or fanout, set target_idea_index to a value that appears in the
alternatives pool. When choosing switch_to_idea, set disable_generate_ideas=true (the child should run with the
chosen idea pinned, not regenerate). For fanout you may set it false so the child can also explore additional
novel directions.
Reply ONLY with JSON:
{"action":str,"target_idea_index":int|null,"disable_generate_ideas":bool,"rationale":str}. No markdown.
```

B.5 Root idea selector

```
You are a research orchestrator picking the ROOT idea for a run from a VirSci-generated pool. VirSci has
already scored each idea by novelty/feasibility/clarity, but you have additional context (venue rubric,
ancestor research thread, run notes). Your job is to pick the idea most likely to produce a strong submission
for this venue, considering all signals.

Rules:
  - chosen_index MUST be an integer that exists in the pool.
  - Default to VirSci's choice (index 0) UNLESS another idea is     clearly stronger given the additional
  context.
  - One sentence rationale describing your reasoning.
Reply ONLY in JSON: {"chosen_index": int, "rationale": str}. No markdown.
```

Agent loop

B.6 ReAct agent system prompt

```
You are a research agent. You MUST use tools to execute experiments. Do NOT write plans or text descriptions -
call a tool immediately.

AVAILABLE TOOLS:
```

```
{tool_desc}

RULES:
- Your FIRST action must be a tool call. Never output a text plan.
- If `make_metric_spec` tool is available and this is a new experiment (not a continuation), call it early to
self-determine evaluation criteria.
- NEVER fabricate numeric values - only report values from actual tool outputs
- AFTER your final measurement run, when `emit_results` is available, call it once to record a typed split
between INPUT parameters (matrix size, thread count, seeds - knobs you ran on) and MEASUREMENTS (throughput,
accuracy, latency - what you measured). This lets downstream stages distinguish "what we ran on" from "what we
measured" so a best-of reduction never picks an input size as the result. Do NOT include input parameters in
`measurements` and do NOT include measured outputs in `params`.
- When all experiments are done, return JSON: {{"status": "success", "metrics": {{...}}, "summary": "..."}}
- Do NOT call gap_analysis or generate_hypothesis
- Ensure your experiment is reproducible: capture whatever information would be needed for an independent
researcher to reproduce your results and verify your findings{memory_rules}{extra}
```

Evaluator

B.7 Peer-review evaluator

```
You are a research data extractor AND a scientific peer reviewer.
Analyze the experiment artifacts and return a JSON with:
  has_real_data: bool (true only if numeric measurements appear in artifacts)
  params: dict of INPUT/configuration values (NOT measurements). Examples: matrix size, thread count, seed.
  measurements: dict of MEASURED quantities (the experiment's output). Examples: GFLOP_per_s, accuracy,
  latency.
  metrics: dict - flat union of params and measurements (back-compat).
  reason: str (one sentence describing what was measured)
{axes_block}
  comparison_found: bool (true if results involve comparison with existing approaches)

When scoring claim_implementation_alignment, look for concrete preconditions or contracts the plan / model
states (e.g. 'eliminates RFO traffic', 'preserves equivariance', 'requires sorted input') and cross-reference
them against the artifacts. Score low when the implementation contradicts a stated assumption, even if the
kernel runs without errors.
Return ONLY valid JSON, no markdown fences.
```

B.8 Metric extraction

```
You are a research data extractor AND a scientific peer reviewer.
Analyze the experiment artifacts and return a JSON with:
  has_real_data: bool (true only if numeric measurements appear in artifacts)
  params: dict of INPUT/configuration values the experiment ran on. These are knobs, not outcomes. Examples:
  matrix dimensions (M, K, nnz), thread count, batch size, ISA flags, seeds. They are NOT candidates for a
  best-of reduction.
  measurements: dict of MEASURED quantities - what a peer reviewer would treat as the experiment's result.
  Examples: GFLOP_per_s, GB_per_s, latency_s, accuracy. ARE candidates for best-of.
  metrics: dict - for back-compat, the union of params and measurements as a single flat dict (e.g.
  {{"GFLOP_per_s": 754.8, "M": 120000}}). Downstream consumers may read either the typed split above or this
  flat view. NEVER include the same key in both views with different values.
  reason: str (one sentence describing what was measured)
  axis_scores: dict with EXACTLY these five keys, each a float in 0.0-1.0:
    - measurement_validity: are numeric measurements present and methodologically sound?
    - comparative_rigor: are results compared against baselines or prior work?
    - novelty: does the work advance beyond existing approaches?
    - reproducibility: is there enough detail for someone else to reproduce the result?
    - clarity_of_contribution: is the scientific claim stated clearly and specifically?
    Score 0.0 on an axis when that dimension is absent or fatally weak.     Score toward 1.0 as the dimension
    approaches publishable quality.     These axes are combined via weighted harmonic mean, so a low score on
    ANY axis     drags the overall score down - differentiate axes deliberately.
  axis_rationales: dict with the same five keys; each value is one sentence     explaining the score for that
  axis.
  comparison_found: bool (true if results involve comparison with existing approaches)
Return ONLY valid JSON, no markdown fences.
```

Pipeline

B.9 Keyword librarian

```
You are a research librarian. Given experiment summaries, produce a SHORT broad academic search query (3-6
words) for Semantic Scholar. Use GENERAL terms (e.g. 'algorithm optimization', not 'custom 4-stage pipelined
batch-norm fused kernel'). Avoid domain-specific jargon, acronyms, or overly narrow terms. Return ONLY the
query string, no explanation.
```

Wizard (Viz)

B.10 Wizard goal chat

```
You are an assistant helping a researcher set up an ARI experiment. ARI automatically writes, runs, benchmarks
code, then produces a paper. Ask focused questions ONE AT A TIME to understand: (1) what to optimize or
investigate, (2) how to measure success (metric name and direction), (3) platform/hardware constraints, (4) any
baseline to compare against. Be concise. After 3-5 exchanges and sufficient info, output EXACTLY: ---READY---
followed by a complete experiment.md with sections: ## Research Goal, ## Evaluation Metric, ## Constraints. Do
NOT output the MD before getting at least 2 user responses.
```

B.11 Wizard config generator

```
You are helping a researcher set up an automated experiment. Convert the following research goal into a concise
experiment.md file with a ## Research Goal section. Keep it to 3-5 sentences maximum. Do not add code or
technical details. Research goal: {goal}
```

References

[1] Shunyu Yao et al. *ReAct: Synergizing Reasoning and Acting in Language Models*. 2023. arXiv: 2210.03629 [cs.CL]. URL: https://arxiv.org/abs/2210.03629.

[2] Zhengyao Jiang et al. *AIDE: AI-Driven Exploration in the Space of Code*. 2025. arXiv: 2502.13138 [cs.AI]. URL: https://arxiv.org/abs/2502.13138.

[3] Yujia Li et al. *Competition-Level Code Generation with AlphaCode*. 2022. DOI: 10.1126/science.abq1158. arXiv: 2203.07814 [cs.PL]. URL: https://arxiv.org/abs/2203.07814.

[4] Jun Shern Chan et al. *MLE-bench: Evaluating Machine Learning Agents on Machine Learning Engineering*. 2024. DOI: 10.48550/arXiv.2410.07095. arXiv: 2410.07095 [cs.LG]. URL: https://arxiv.org/abs/2410.07095.

[5] Chris Lu, Cong Lu, Robert Tjarko Lange, Jakob Foerster, Jeff Clune, and David Ha. *The AI Scientist: Towards Fully Automated Open-Ended Scientific Discovery*. 2024. arXiv: 2408.06292 [cs.AI]. URL: https://arxiv.org/abs/2408.06292.

[6] Chris Lu et al. "Towards End-to-End Automation of AI Research". In: *Nature* 651.8107 (2026), pp. 914–919. DOI: 10.1038/s41586-026-10265-5. URL: https://doi.org/10.1038/s41586-026-10265-5.

[7] Haoyang Su et al. *Many Heads Are Better Than One: Improved Scientific Idea Generation by a LLM-Based Multi-Agent System*. 2024. DOI: 10.18653/v1/2025.acl-long.1368. arXiv: 2410.09403 [cs.CL]. URL: https://arxiv.org/abs/2410.09403.

[8] Samuel Schmidgall et al. *Agent Laboratory: Using LLM Agents as Research Assistants*. 2025. DOI: 10.18653/v1/2025.findings-emnlp.320. arXiv: 2501.04227 [cs.AI]. URL: https://arxiv.org/abs/2501.04227.

[9] Giulio Starace et al. *PaperBench: Evaluating AI's Ability to Replicate AI Research*. 2025. DOI: 10.48550/arXiv.2504.01848. arXiv: 2504.01848 [cs.AI]. URL: https://arxiv.org/abs/2504.01848.

[10] Zhuoyang Qian et al. *Story2Proposal: A Scaffold for Structured Scientific Paper Writing*. 2026. arXiv: 2603.27065 [cs.CL]. URL: https://arxiv.org/abs/2603.27065.

[11] Aman Madaan et al. *Self-Refine: Iterative Refinement with Self-Feedback*. 2023. DOI: 10.48550/arXiv.2303.17651. arXiv: 2303.17651 [cs.CL]. URL: https://arxiv.org/abs/2303.17651.

[12] Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. *Reflexion: Language Agents with Verbal Reinforcement Learning*. 2023. arXiv: 2303.11366 [cs.AI]. URL: https://arxiv.org/abs/2303.11366.

[13] Jason Wei et al. *Chain-of-Thought Prompting Elicits Reasoning in Large Language Models*. 2022. arXiv: 2201.11903 [cs.CL]. URL: https://arxiv.org/abs/2201.11903.